Operating Systems

[1500WETOPS]

José Oramas

Disk Management

[How to Access Information on Disk Drives]

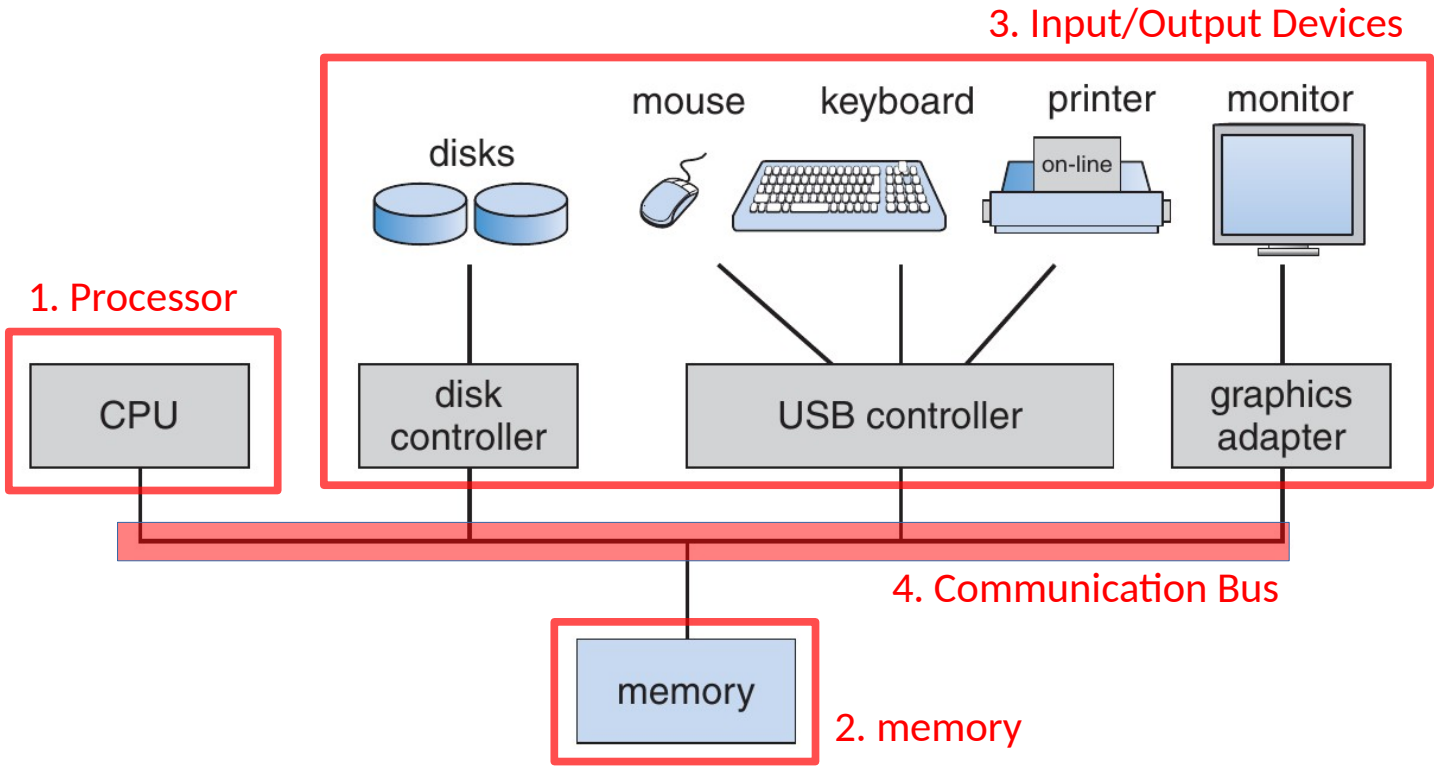
José Oramas

University of Antwerp

Operating systems

A Modern Computer System

- Main components



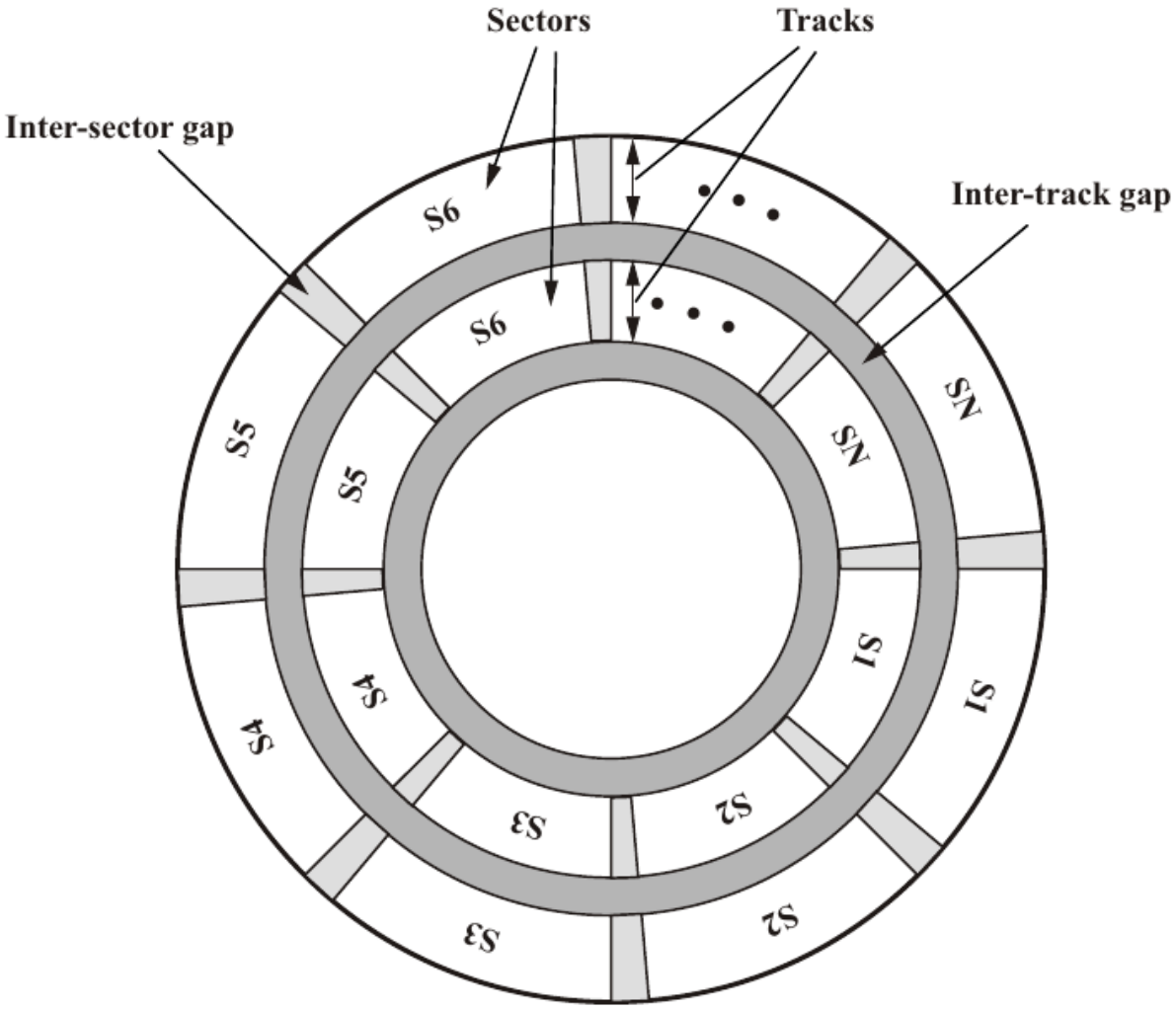
Disk Organization

Disk components/operation

- One or more circular (metal) disks
- Disk head (per disk)
- Fixed rotation speed (7200, 10000 rpm)

Disk partitioning

- Tracks (concentric circles)
- Sectors (fixed number of bytes): unit of reading and write operations

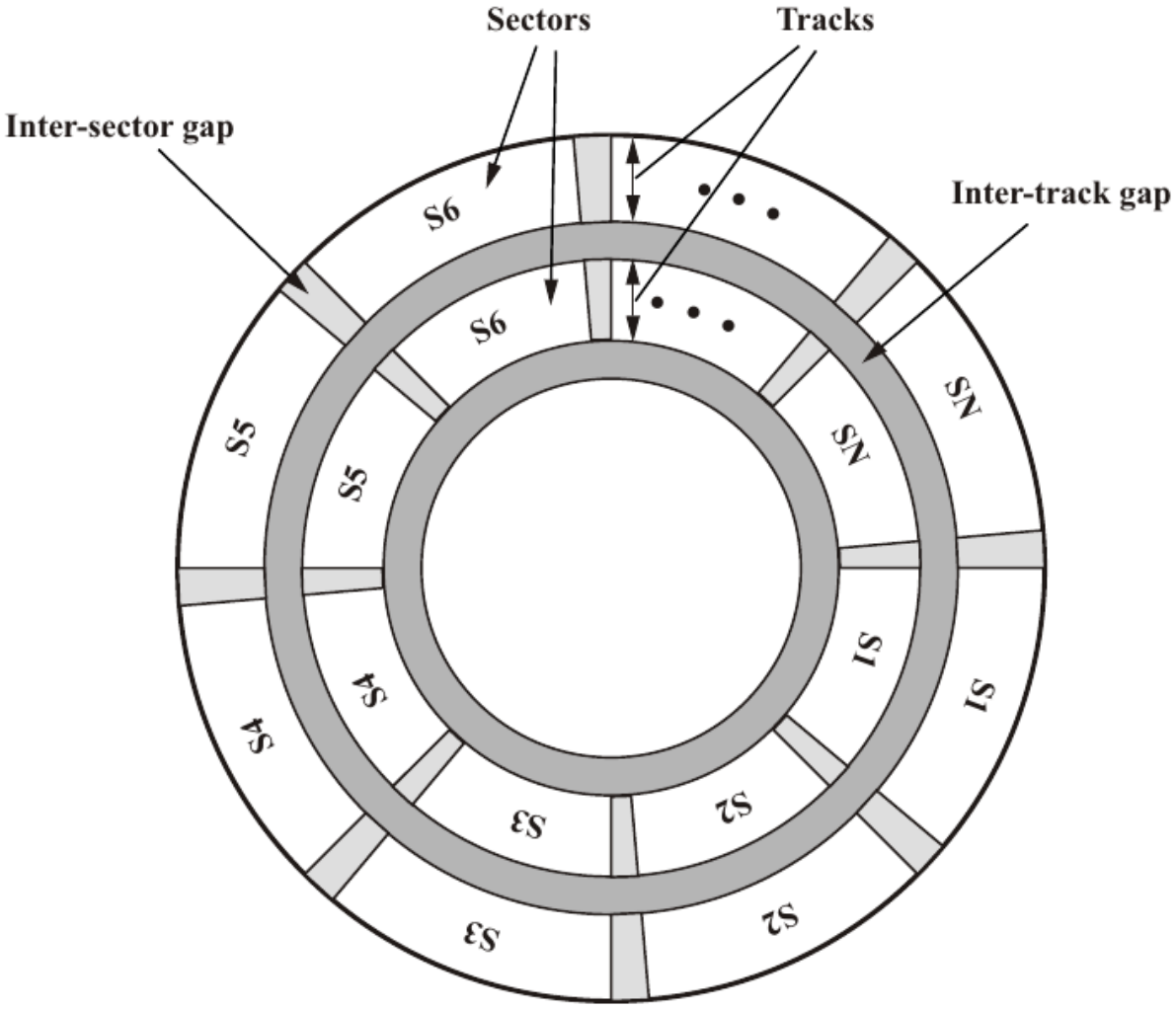


Disk Organization

Classical Layout

- Fixed number sectors per track
- Compact storage near the center

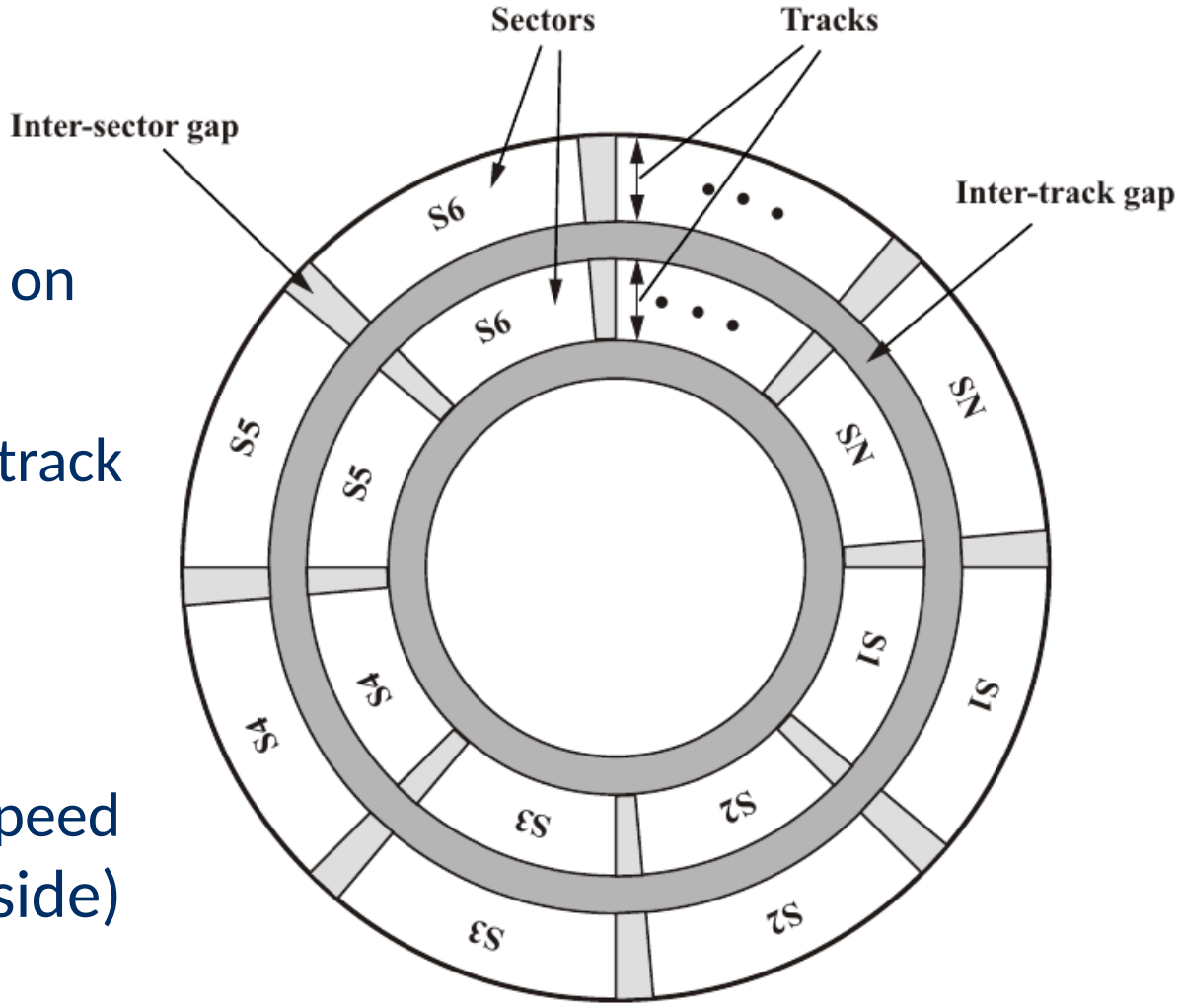
  [high density]
- Low density on the outside
- Fixed read and write speed



Disk Organization

Zone Bit Recording (ZBR)

- Modern discs divided into zones (~15):
  - Within a zone: fixed number of sectors on each track
  - Accross zones: Number of sectors per track varies (decreases towards the inside)
- In practice: up to 2 times as many sectors in outer zone (several hundreds)
- Result: almost double reading and writing speed on the outside (filling from outside to inside)



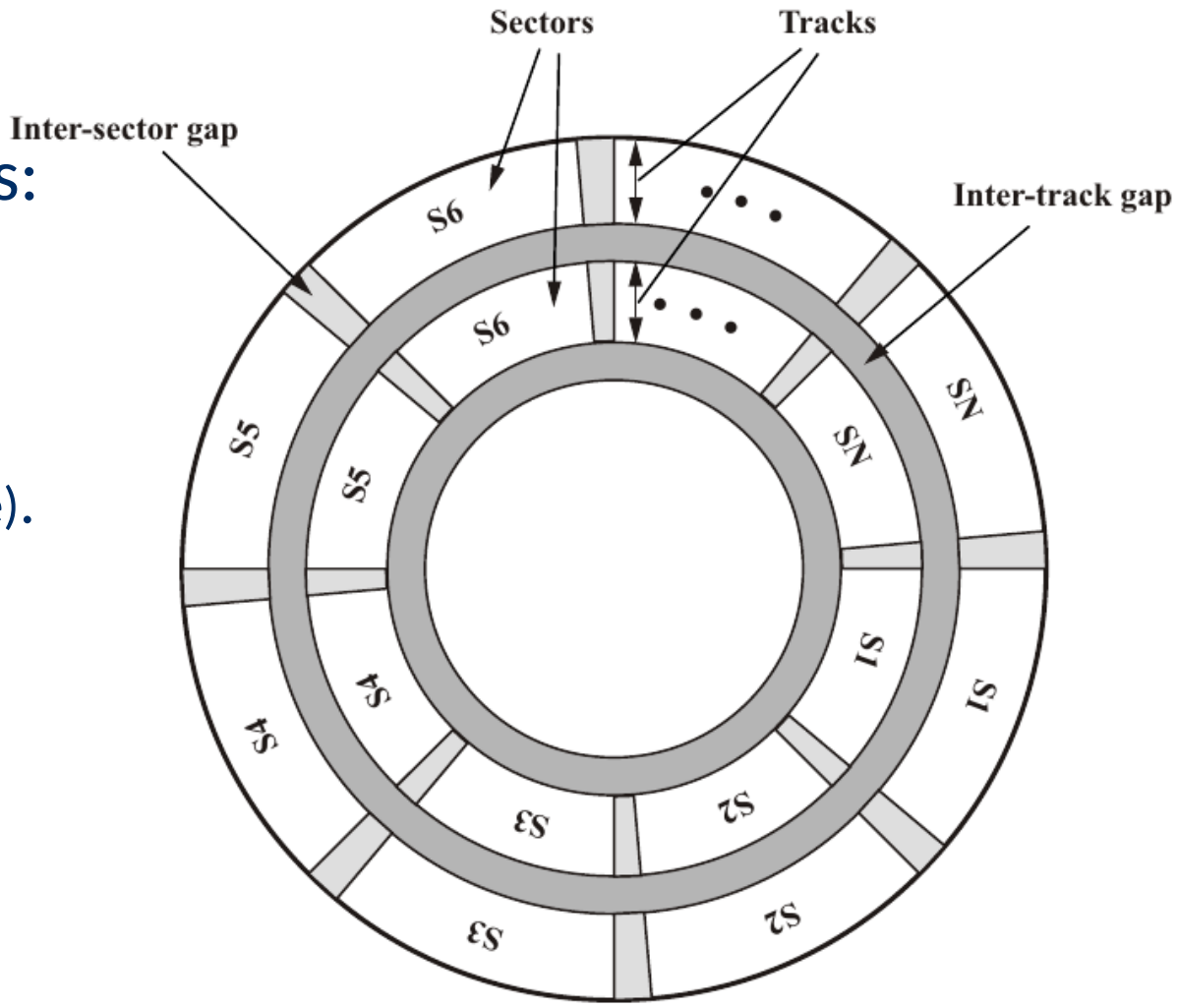
Disk Management

Access Times

- Access time T_a (for b bytes) in seconds:

  $T_a = T_s + 1/(2r) + b/(rN)$,

  - with N bytes per track,
  - T_s: positioning of the head (seek time).
  - r: spinning speed



Disk Management

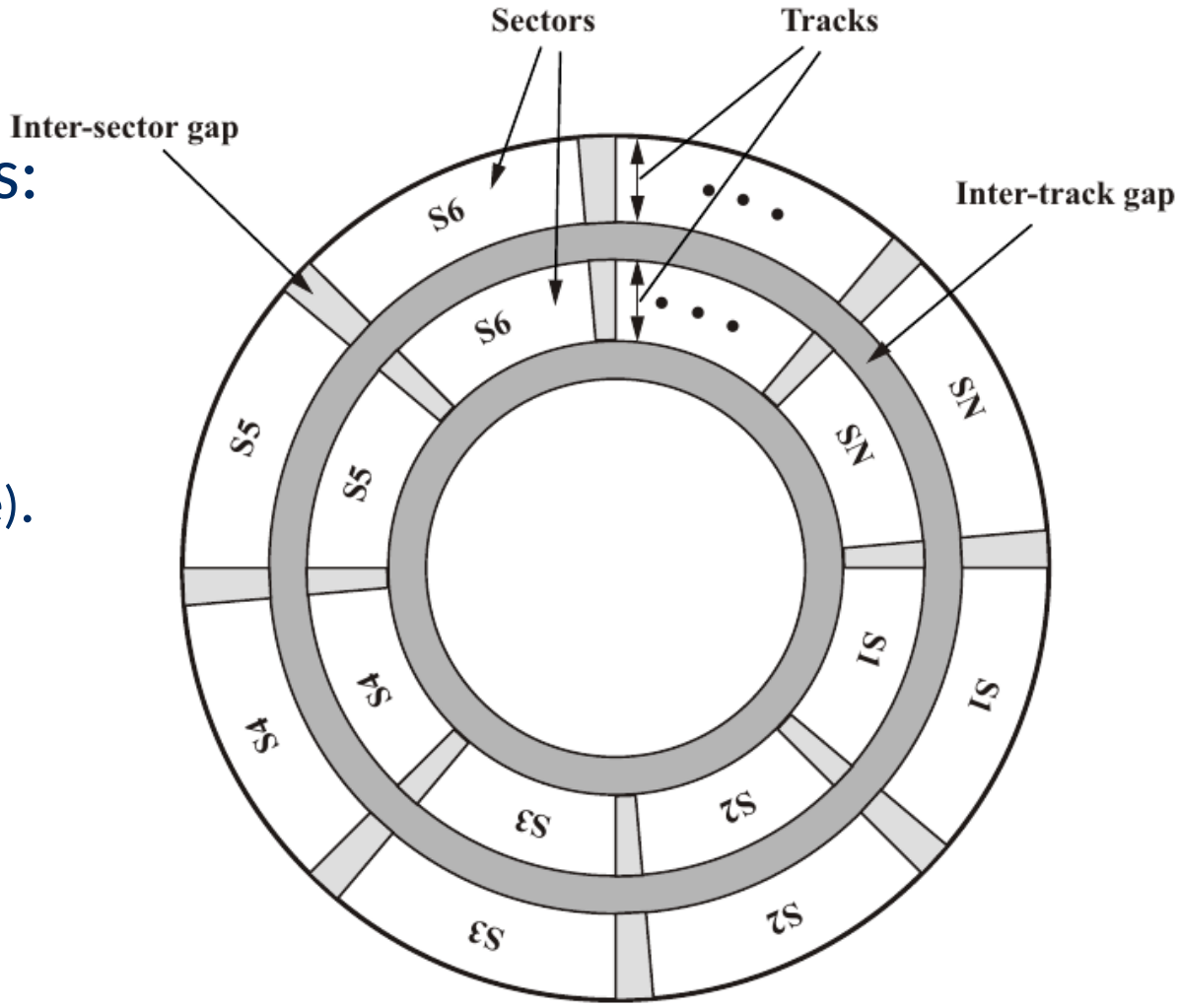
Access Times

- Access time T_a (for b bytes) in seconds:

  $T_a = T_s + 1/(2r) + b/(rN)$,

  - with N bytes per track,
  - T_s: positioning of the head (seek time).
  - r: spinning speed

Q: Why is the position of the head insufficient for I/O?



Disk Management

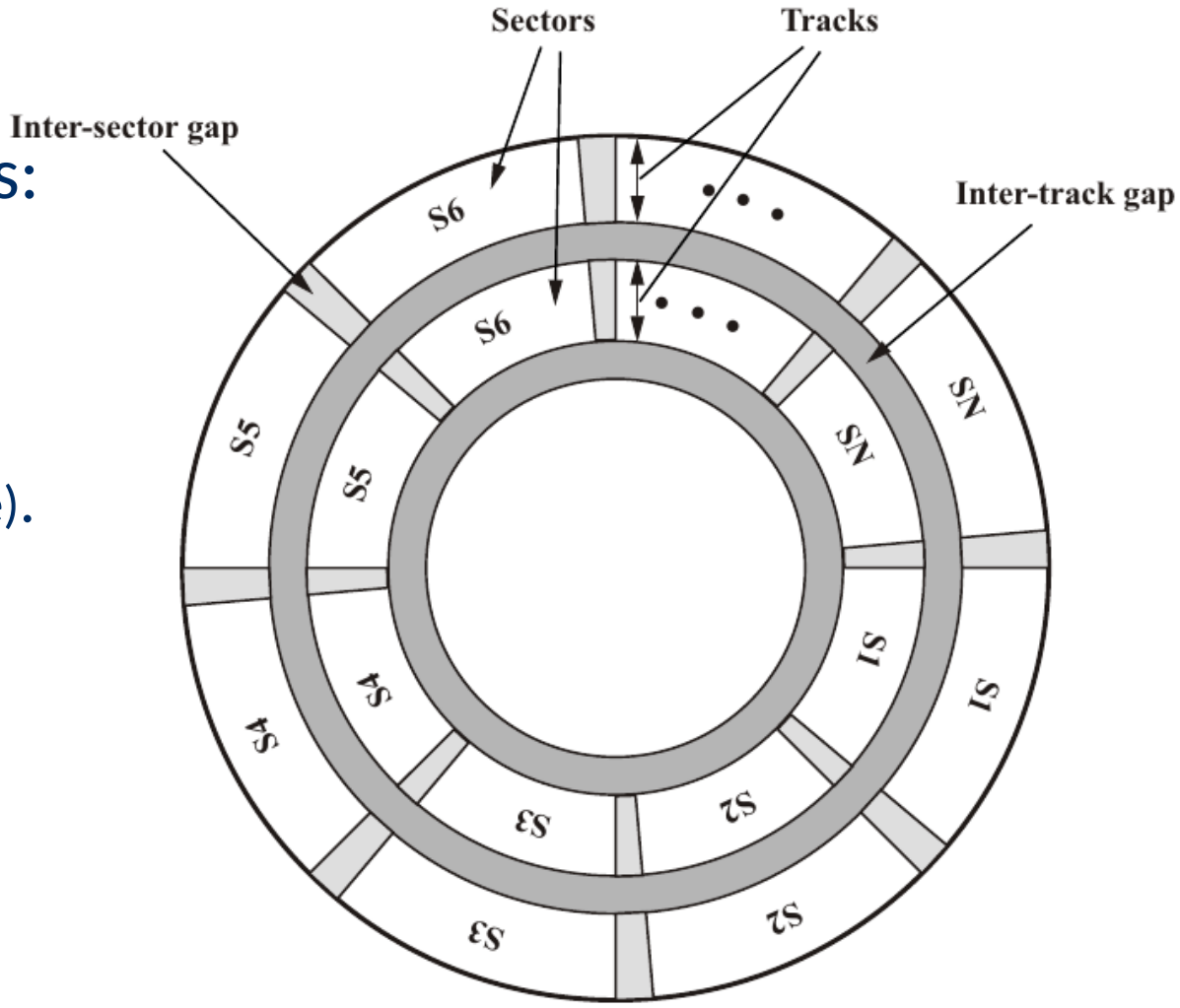
Access Times

- Access time T_a (for b bytes) in seconds:

  $T_a = T_s + 1/(2r) + b/(rN)$,

  - with N bytes per track,
  - T_s: positioning of the head (seek time).
  - r: spinning speed

Q: Why is the position of the head insufficient for I/O?



Disk Management

Access Times

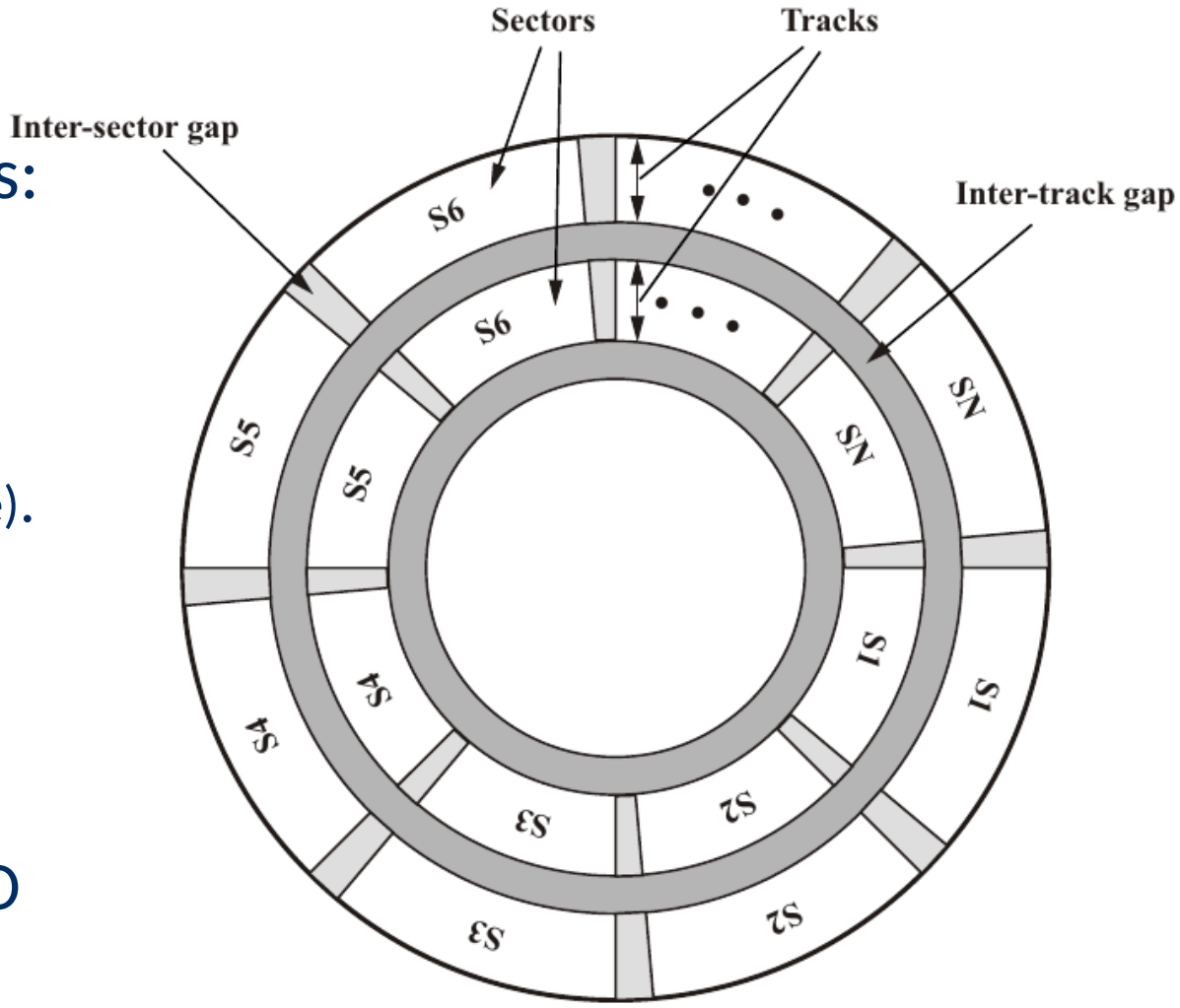
- Access time T_a (for b bytes) in seconds:

  $$T_a = T_s + 1/(2r) + b/(rN),$$

  - with N bytes per track,
  - T_s: positioning of the head (seek time).
  - r: spinning speed

  Q: Why is the position of the head insufficient for I/O?

- Only valid for random reading, since multiple read and write operations in I/O buffers → slightly different order



Disk Management

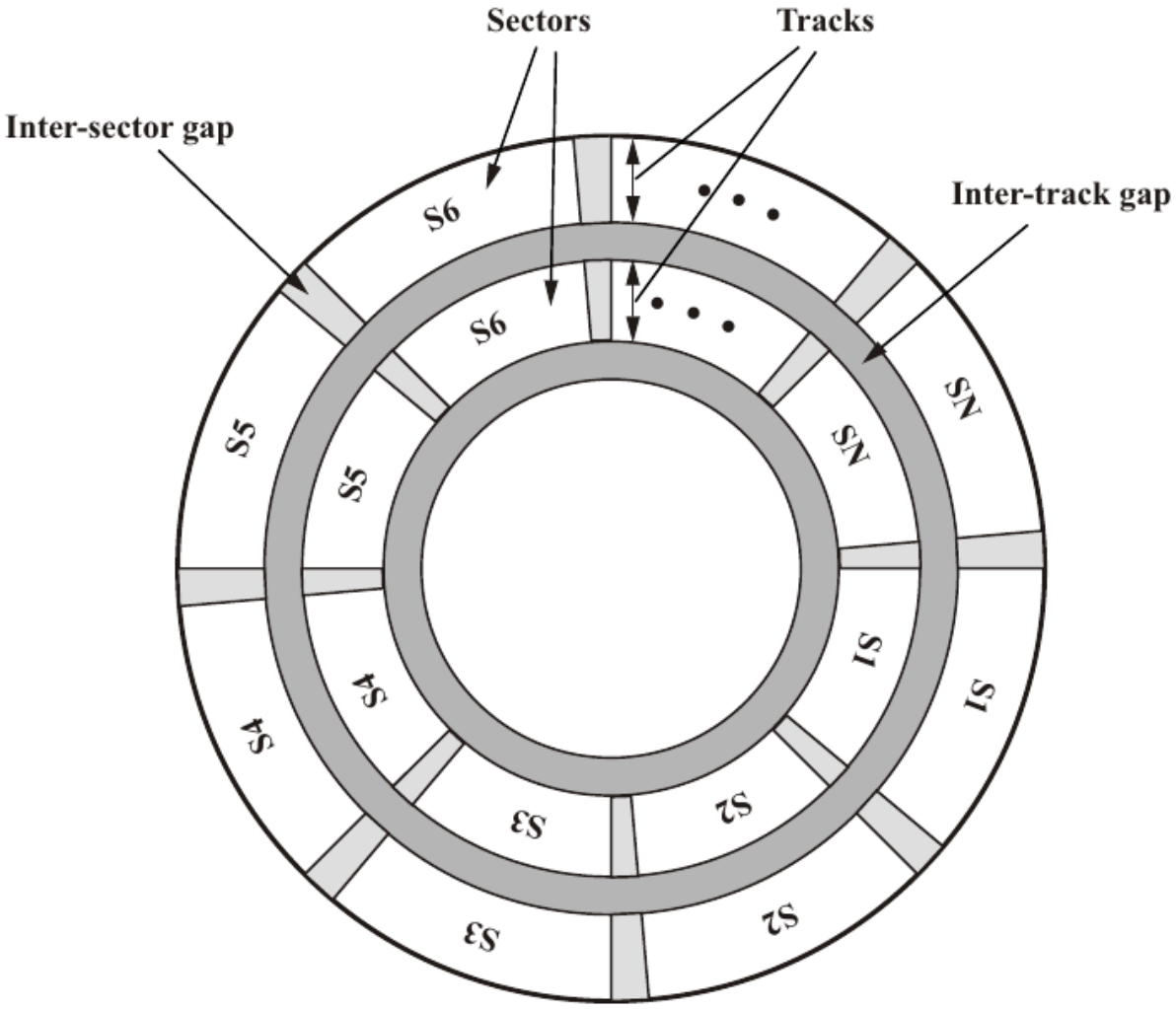
Seek Times

- Composed by four phases:

  [speed-up, coast, slowdown and settle]

- Example:
  - Very short seeks (~2-4 tracks) → mainly settle
  - Short seeks (~100 tracks) → no coast
  - Long Seeks → mostly coast.

    (time increases linearly with number of tracks)



Disk Management

Seek Times

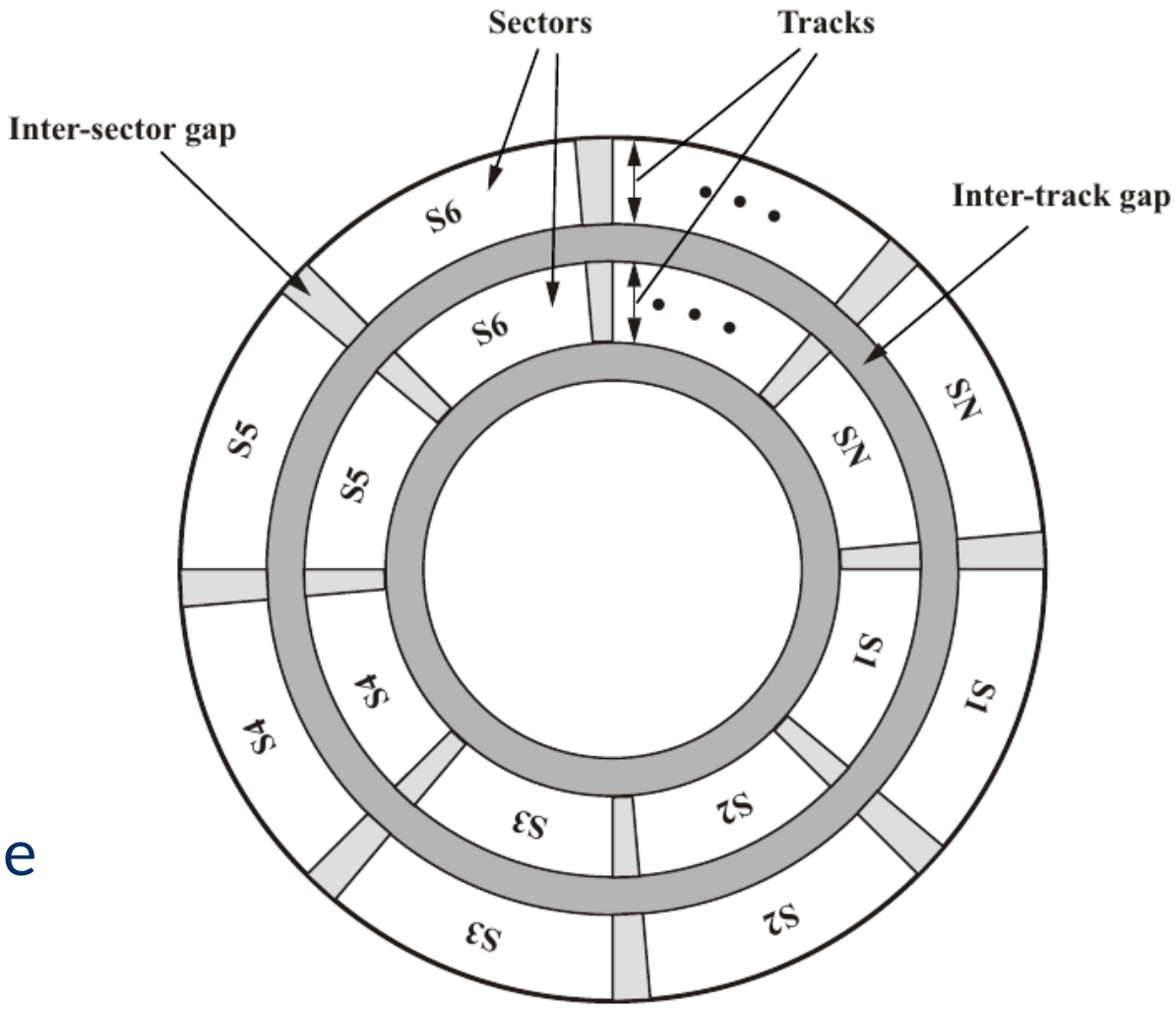
- Composed by four phases:

  [speed-up, coast, slowdown and settle]

- Example:
  - Very short seeks (~2-4 tracks) → mainly settle
  - Short seeks (~100 tracks) → no coast
  - Long Seeks → mostly coast.

    (time increases linearly with number of tracks)

- Same track with multiple discs: also settle time (disc not perfectly equal)

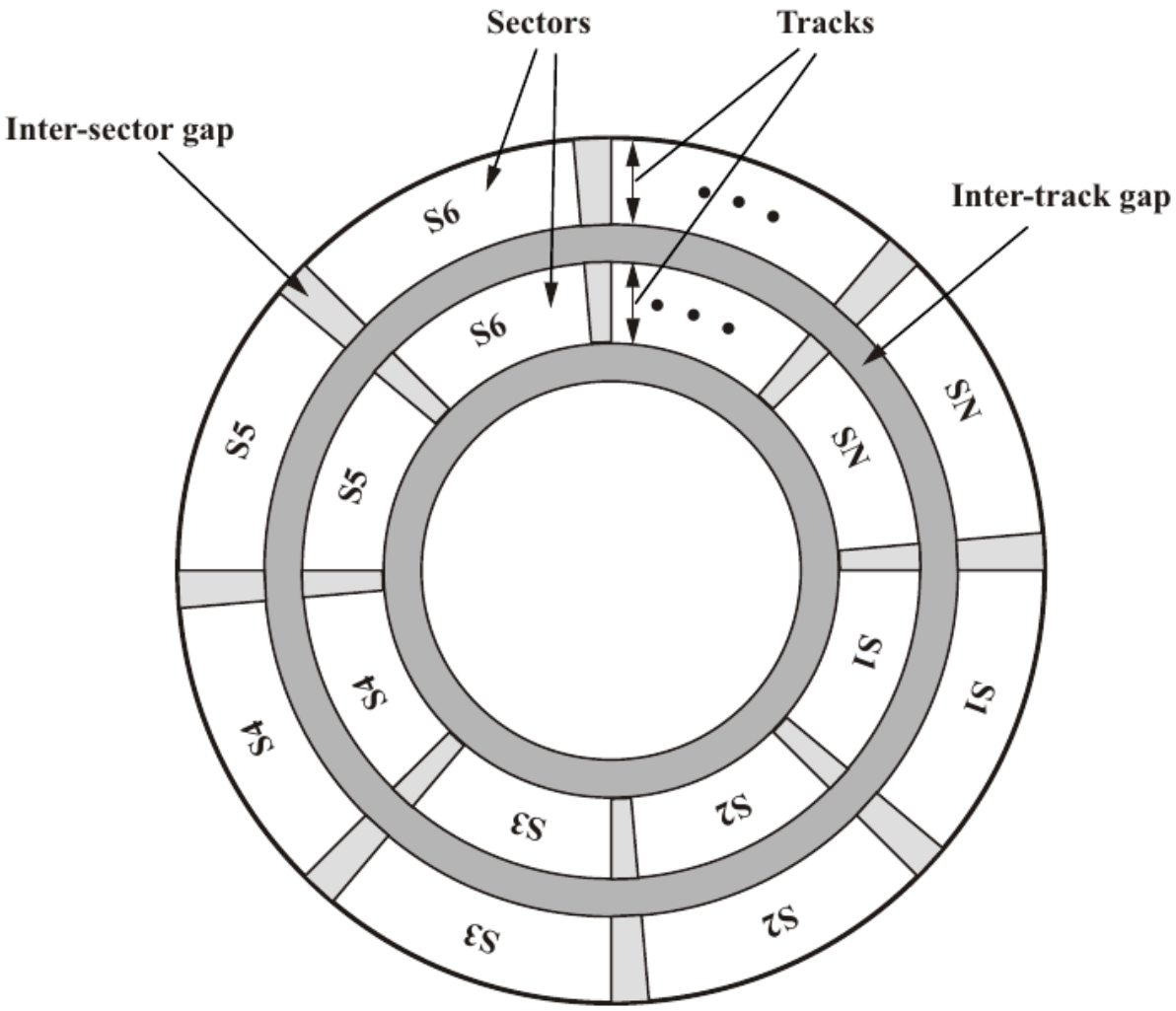


Disk Management

Seek Times

- Disk controller determines power of motor driving arm
- Power values stored in a table

  [interpolation is enabled]
- Table requires regular (hourly) recalibration due to humidity, temperature, etc.

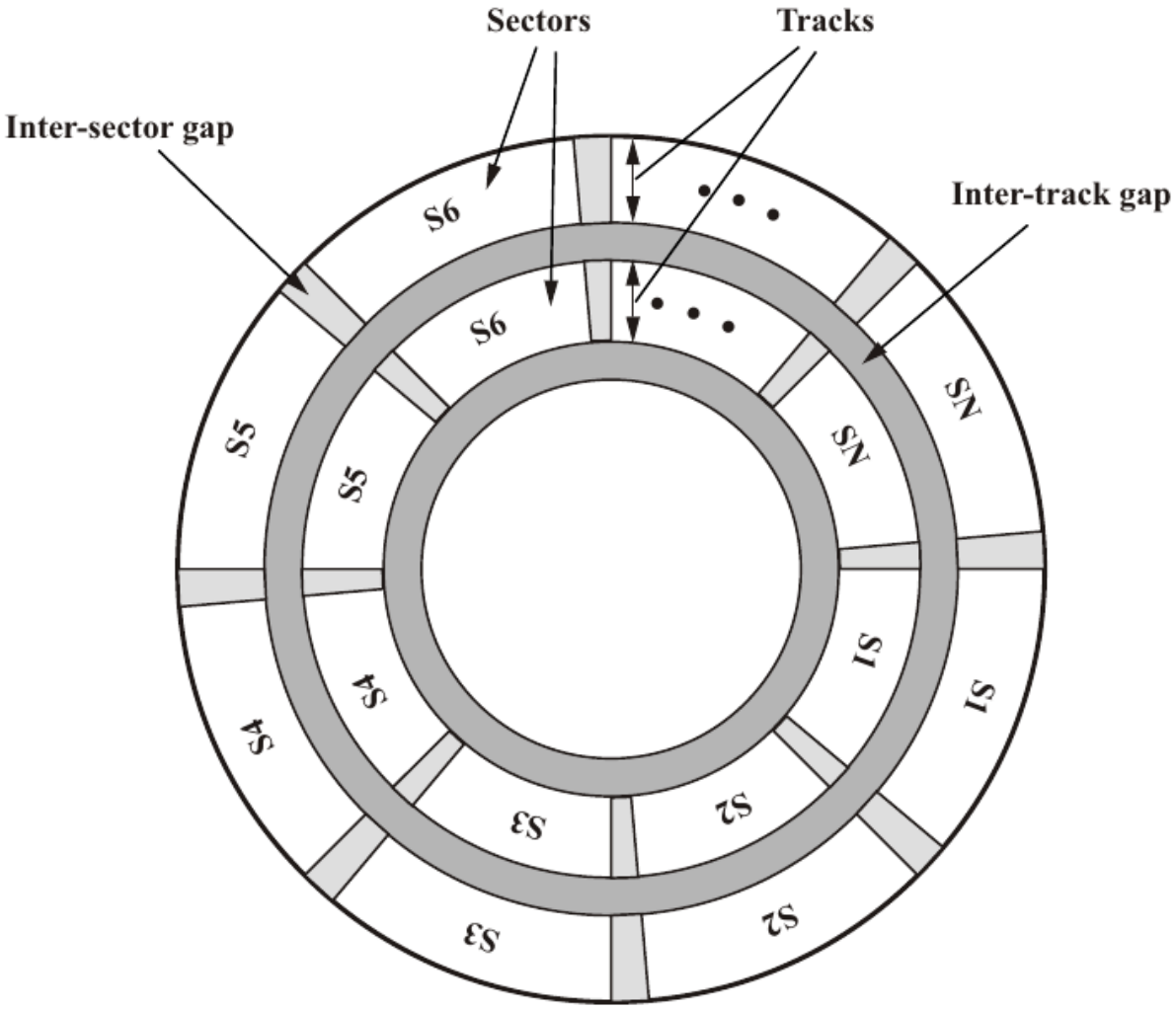


Disk Management

Skewing

- Goal: keep read speed high.
- Shift sectors on consecutive tracks somewhat(worst case track change)

  → reduce rotational delay

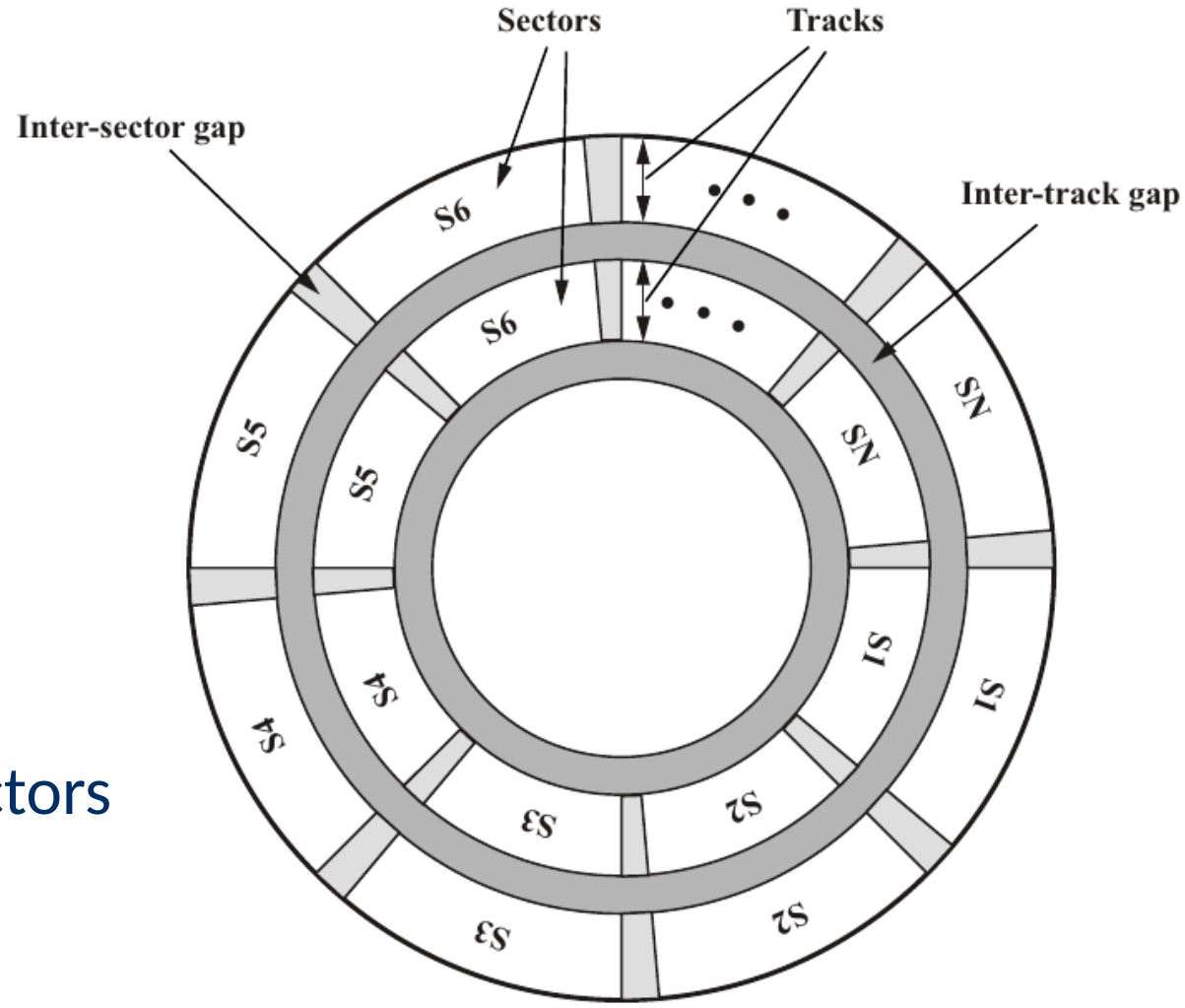


Disk Management

Skewing

- Goal: keep read speed high.
- Shift sectors on consecutive tracks somewhat(worst case track change)

  → reduce rotational delay



Sparing

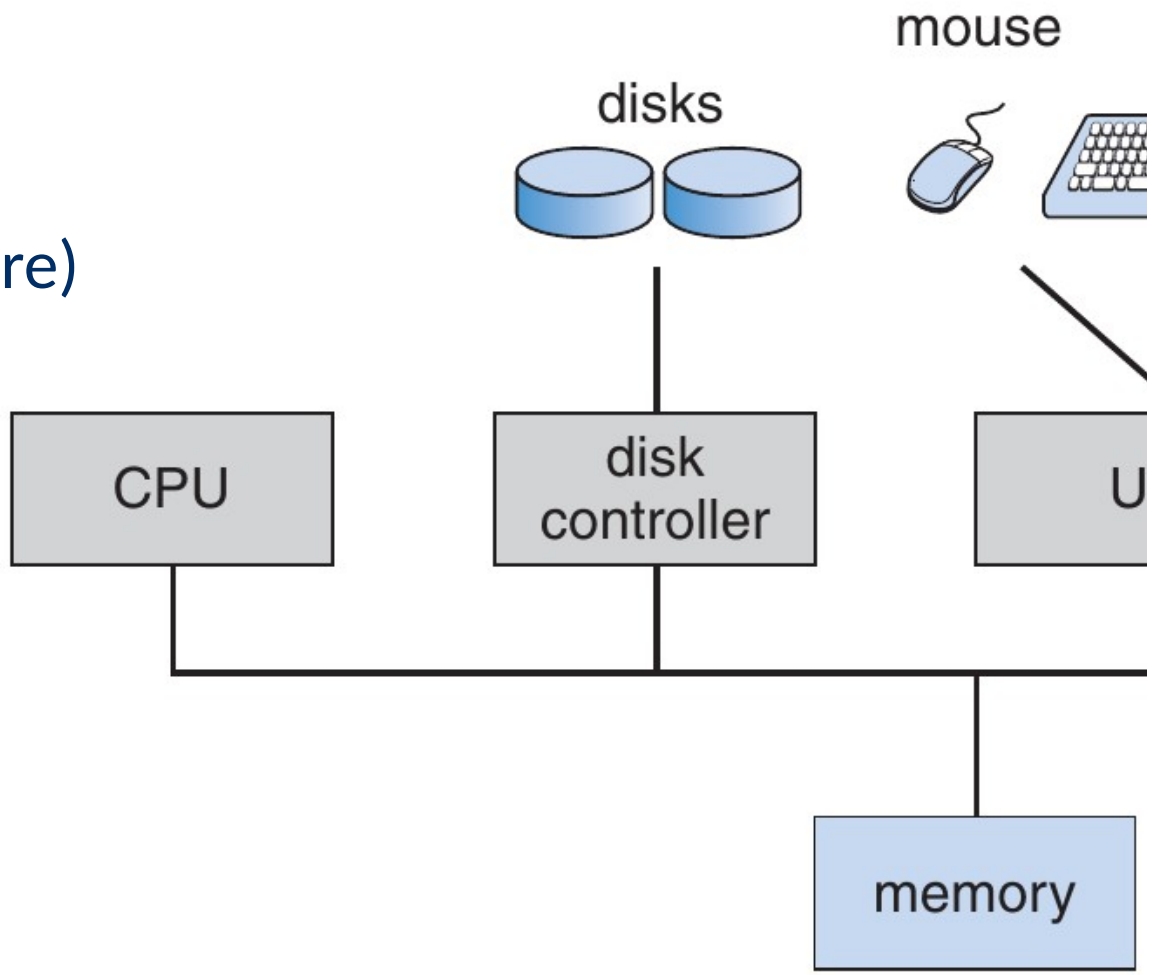
- Map faulty (factory) sectors to other sectors via a list passed to the controller.

Disk Management

Disk Controller

- Connected to the bus
- Receives request (via device driver software) and handles them
- Can buffer at least one sector

  (necessary because of faster bus)
- *Command Queueing*:
  - Controller can receive multiple requests and determine the order of processing.



Disk Management

Disk Caching

- Cache with *read-ahead* strategy
- Read data from cache is replaced immediately

  File reading is usually sequential

Disk Management

Disk Caching

- Cache with *read-ahead* strategy
- Read data from cache is replaced immediately

  File reading is usually sequential
- *Aggressive read-ahead*: over multiple tracks

Disk Management

Disk Caching

- Cache with *read-ahead* strategy
- Read data from cache is replaced immediately

  File reading is usually sequential
- *Aggressive read-ahead*: over multiple tracks
- Old disk: *on-arrival read-ahead*
  - Reading begins when the head is in the right track location.
  - Useful if we usually need the largest part of the track (increasingly less the case)
- Cache useful for quickly writing the same data, but volatile

Disk Scheduling

Disk Scheduling Algorithms

First-In First-Out (FIFO)

- FIFO algorithm
  - Process requests in order of arrival

- Properties
  - Fair
  - Very efficient for clustered operations
  - Inneficient for random reads

Disk Scheduling Algorithms

First-In First-Out (FIFO)

- FIFO algorithm
  - Process requests in order of arrival

- Properties
  - Fair
  - Very efficient for clustered operations
  - Inneficient for random reads

Q: Why is it inneficient for random reads?

Disk Scheduling Algorithms

First-In First-Out (FIFO)

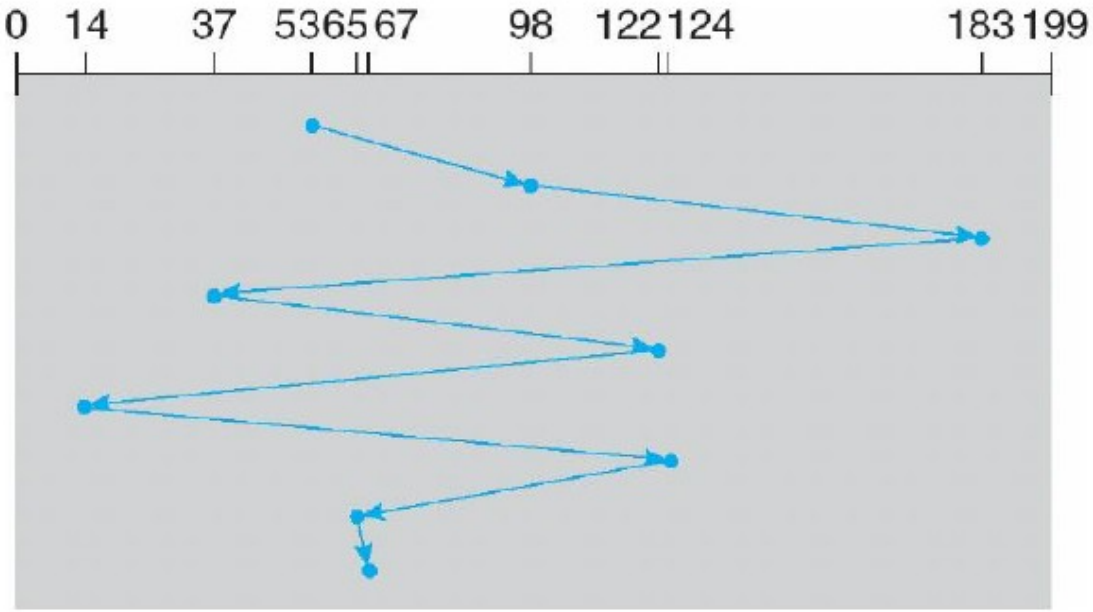
- FIFO algorithm
  - Process requests in order of arrival

- Properties
  - Fair
  - Very efficient for clustered operations
  - Inneficient for random reads

Q: Why is it inneficient for random reads?

Example

– Starting point = 53
– Queue = 98, 183, 37, 122, 14, 124, 65, 67



(a) FIFO

Disk Scheduling Algorithms

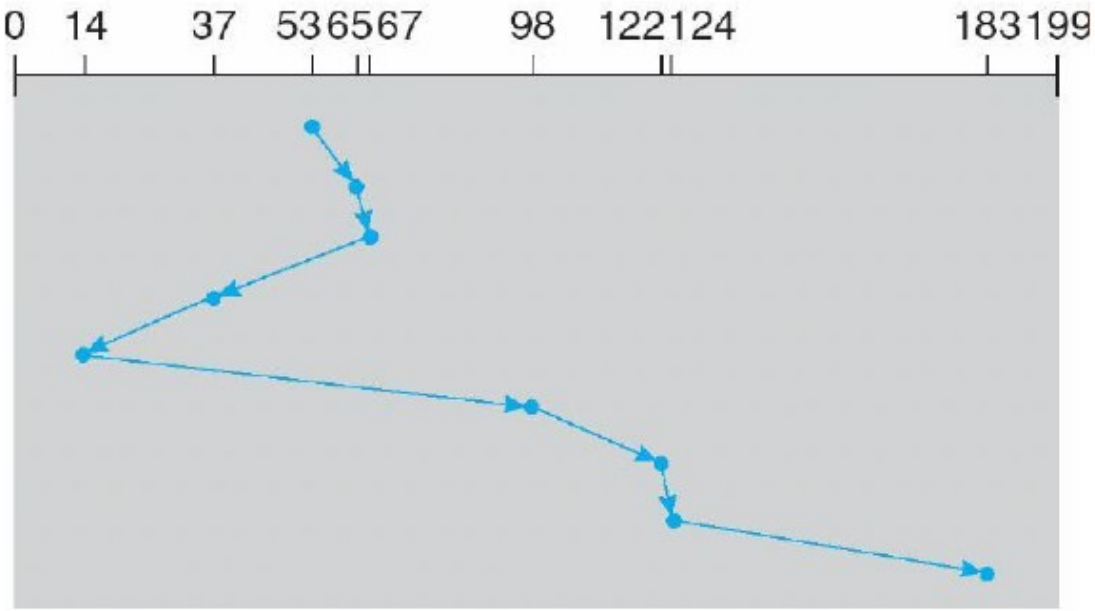
Shortest Seek Time First (SSTF)

- SSTF algorithm
  - Process first request with smallest seek time

- Properties
  - Requires estimation of seek time (non-linear)
  - high utilization/efficiency
    - locality of reference

Example

– Starting point = 53
– Queue = 98, 183, 37, 122, 14, 124, 65, 67



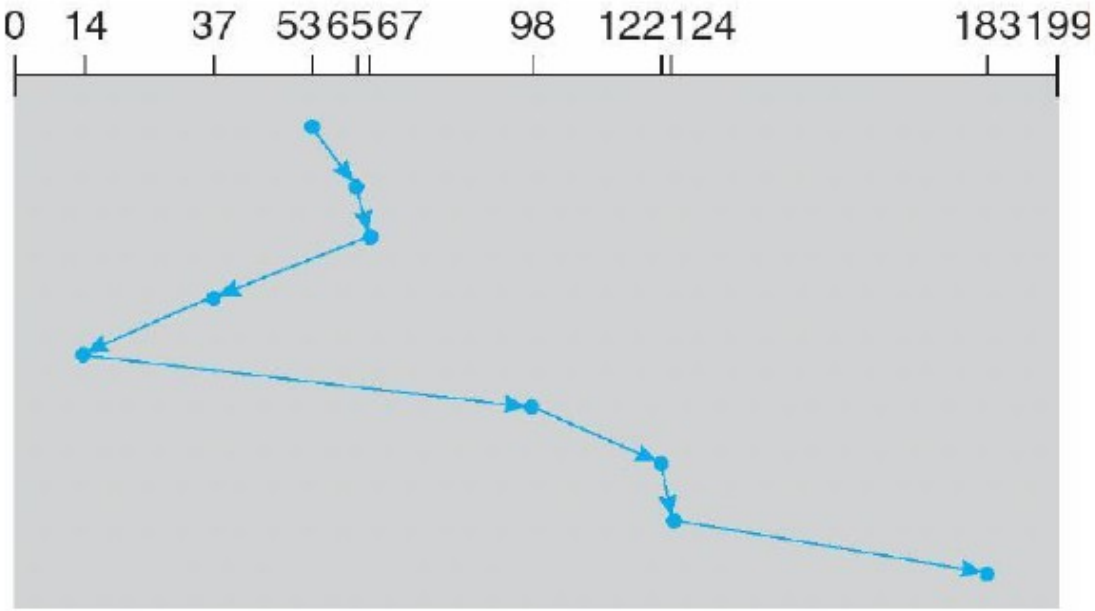
Disk Scheduling Algorithms

Shortest Seek Time First (SSTF)

- SSTF algorithm
  - Process first request with smallest seek time

- Properties
  - Requires estimation of seek time (non-linear)
  - high utilization/efficiency
    - locality of reference

Example
– Starting point = 53
– Queue = 98, 183, 37, 122, 14, 124, 65, 67



Any disadvantage?
– What would happen if suddenly many clustered request occur at the end of the queue?

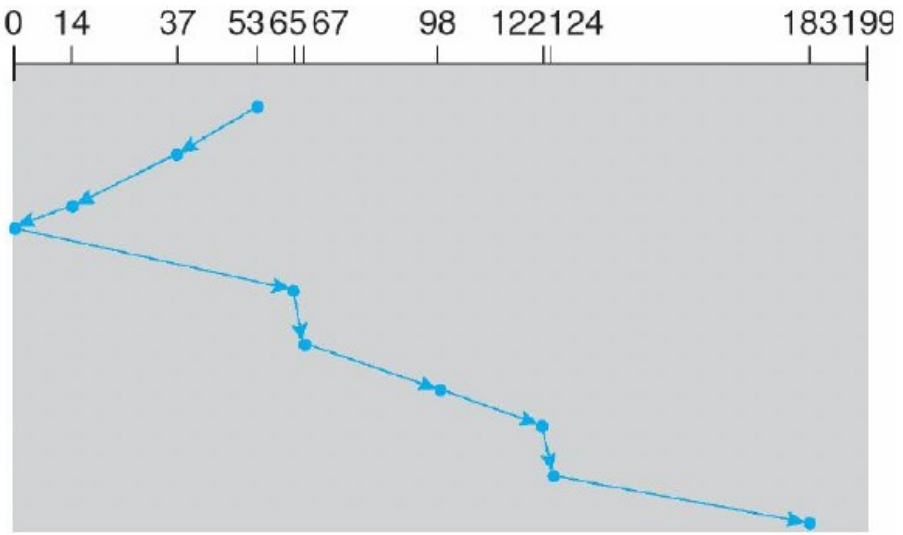
Disk Scheduling Algorithms

SCAN and Cyclic-SCAN (C-SCAN)

- SCAN algorithm
  - SSTF with arm changing direction when reaching the extremes of the disk.

- Properties
  - Less starvation than SSTF
  - Lower delay variance, higher average delay.
  - Unfairness center vs. extreme tracks.

Example

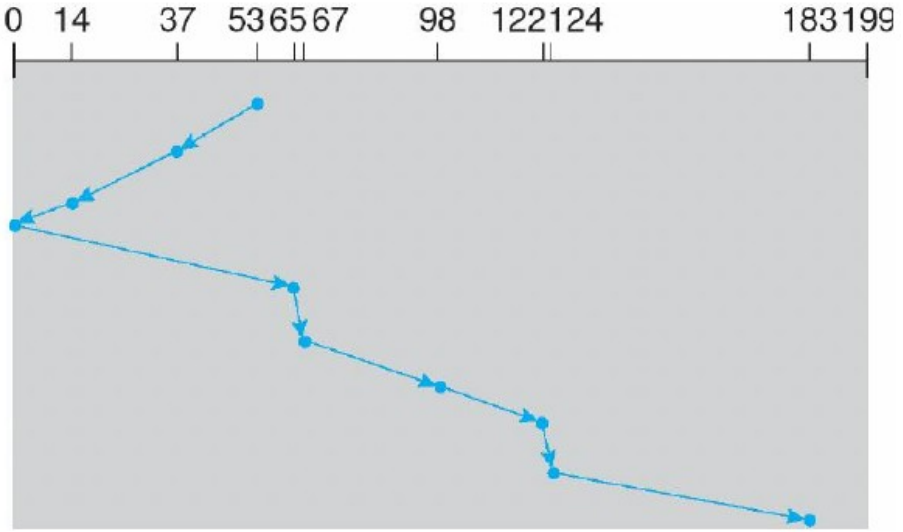


Disk Scheduling Algorithms

SCAN and Cyclic-SCAN (C-SCAN)

- SCAN algorithm
  - SSTF with arm changing direction when reaching the extremes of the disk.
- Properties
  - Less starvation than SSTF
  - Lower delay variance, higher average delay.
  - Unfairness center vs. extreme tracks.
- C-SCAN algorithm
  - SCAN but once a extreme is reached, continue from the other extreme
  - Reduces unfairness

Example



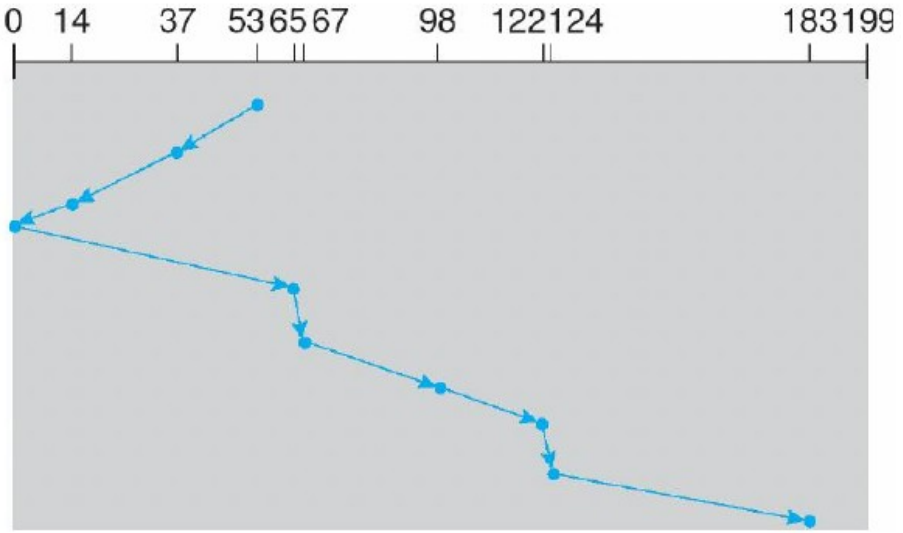
Disk Scheduling Algorithms

LOOK and C-LOOK

- LOOK algorithm
  - SCAN but the arm changes direction if there are no more requests in the current direction.

- Properties
  - Slightly more efficient than SCAN
  - More Complex
  - Unfairness center vs. extreme tracks.

Example

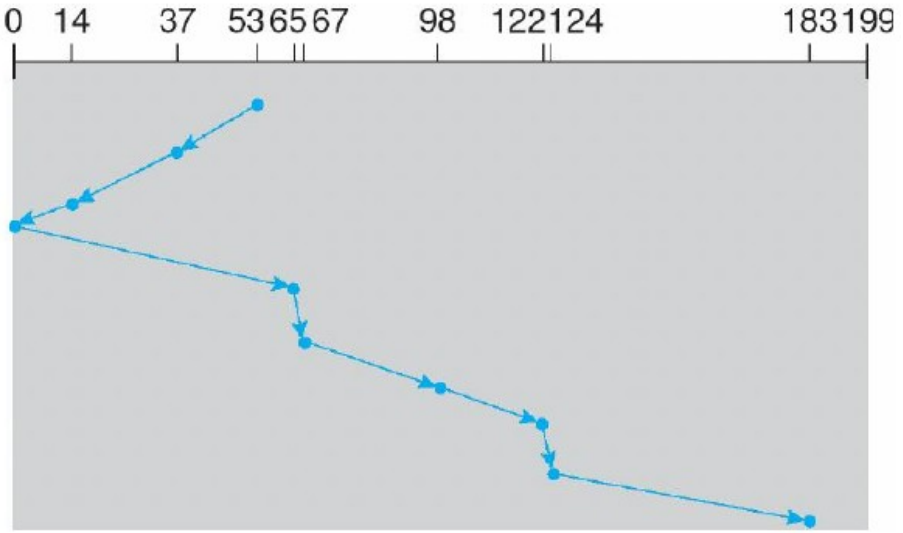


Disk Scheduling Algorithms

LOOK and C-LOOK

- LOOK algorithm
  - SCAN but the arm changes direction if there are no more requests in the current direction.
- Properties
  - Slightly more efficient than SCAN
  - More Complex
  - Unfairness center vs. extreme tracks.
- C-LOOK algorithm
  - Similar operation to C-SCAN.
  - Addresses similar issues.

Example



Disk Scheduling Algorithms

VSCAN and FSCAN

- VSCAN Algorithm
  - Introduces a parameter *R* with $0 \leq R \leq 1$
  - Process first request which is closest located, with:

    distance = SSTF distance + **1**(if changing direction) * *R* * width of the disk (in T_s)

Disk Scheduling Algorithms

VSCAN and FSCAN

- VSCAN Algorithm
  - Introduces a parameter *R* with $0 \leq R \leq 1$
  - Process first request which is closest located, with:
    distance = SSTF distance + **1**(if changing direction) * *R* * width of the disk (in T_s)
- Properties
  - VSCAN(0) = SSTF
  - VSCAN(1) = LOOK
  - VSCAN(0.2) gives good balance ts SSTF and LOOK

Q: any potential weakness so far?

Disk Scheduling Algorithms

VSCAN and FSCAN

- VSCAN Algorithm
  - Introduces a parameter *R* with $0 \leq R \leq 1$
  - Process first request which is closest located, with:
    distance = SSTF distance + **1**(if changing direction) * *R* * width of the disk (in T_s)
- Properties
  - VSCAN(0) = SSTF
  - VSCAN(1) = LOOK
  - VSCAN(0.2) gives good balance ts SSTF and LOOK

Q: any potential weakness so far?

- FSCAN Algorithm
  - Use a two-stage buffer to handle I/O
  - Apply SCAN on the requests in the first buffer, then process requests in the second buffer

RAID

Redundant Arrays of Independent Disks

Redundant Arrays of Independent Disks

RAID – Overview

- Patterson, Gibson & Katz (1988 - SIGMOD).
- Idea:

  Many inexpensive disks form one large, fast and reliable disk

Redundant Arrays of Independent Disks

RAID – Overview

- Patterson, Gibson & Katz (1988 - SIGMOD).
- Idea:

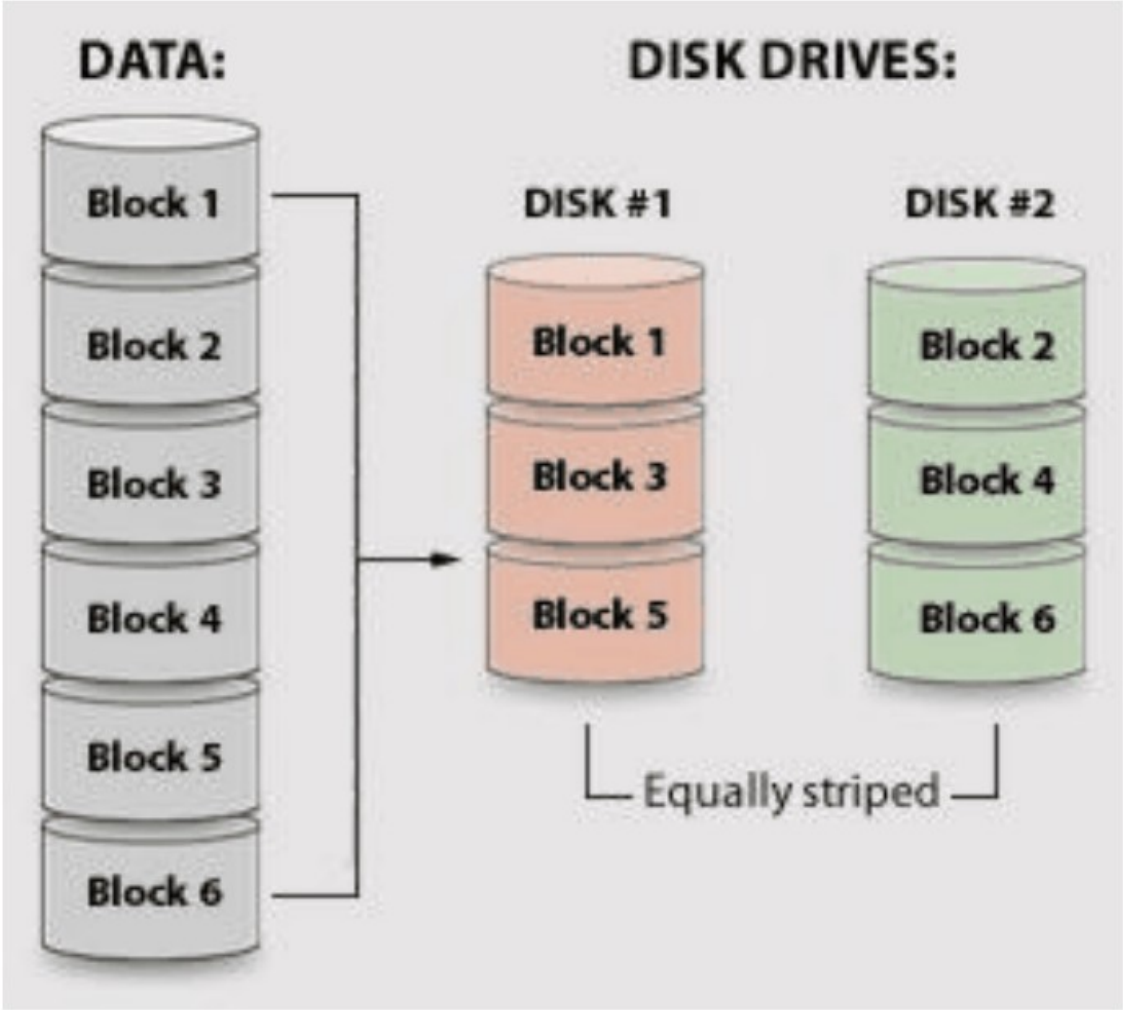
  Many inexpensive disks form one large, fast and reliable disk

- Performance:
  - Read and write speed
  - Random vs. sequential reads/writes
  - Robustness

Redundant Arrays of Independent Disks

RAID 0 (=AID)

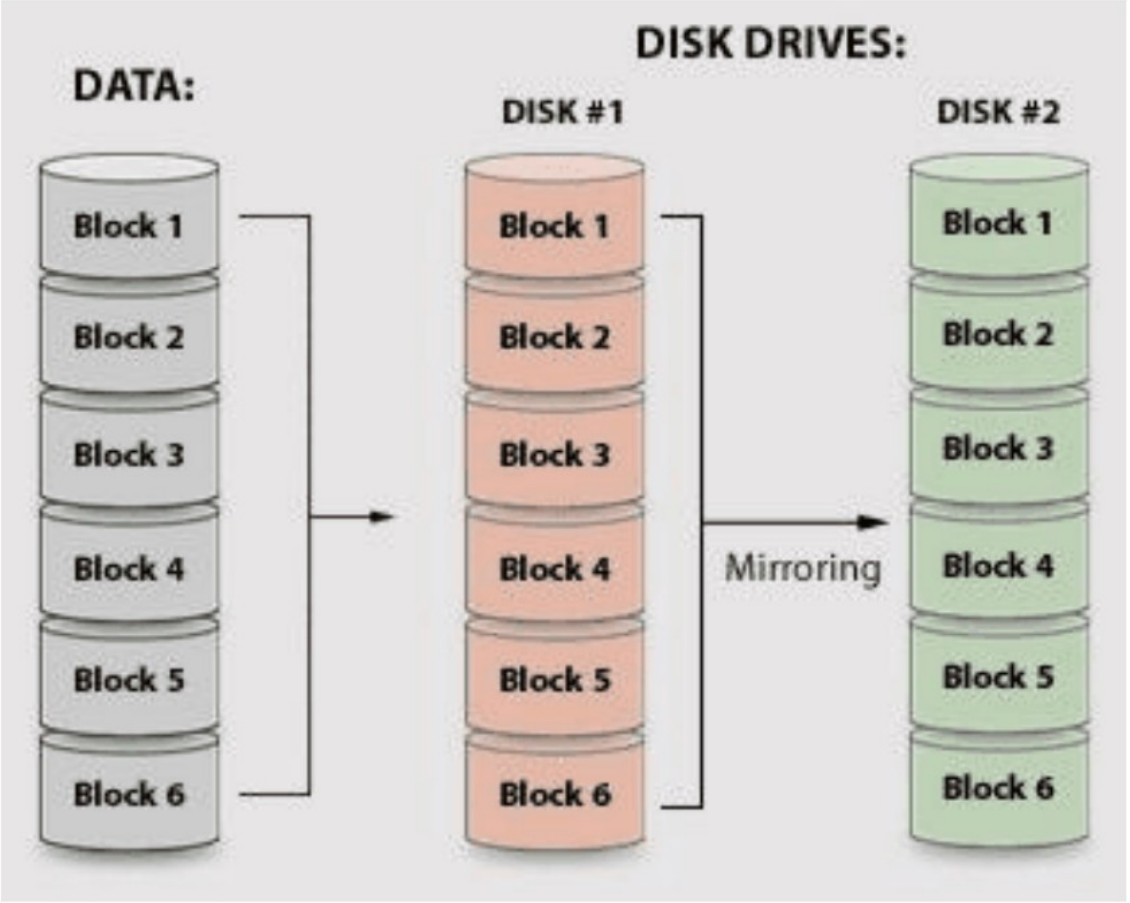
- Data striping over N disks
  - Divide data into blocks
  - Disk *i* contains blocks *i*, *i+N*, *i+2N*, etc.
- *Segment/Block* length (Effect)
  - Large: Random reads simultaneous
  - Small (1024): Sequential read fast (seek & rotation a bit worse)
- Robustness: poor



Redundant Arrays of Independent Disks

RAID 1

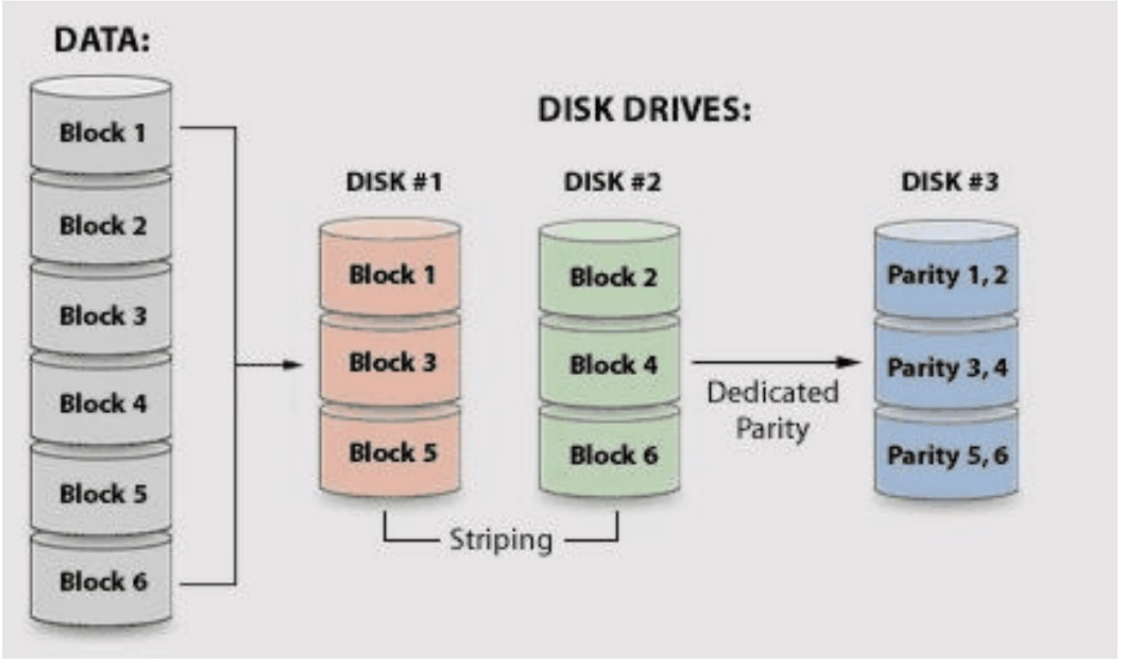
- Data mirroring over 2 discs
  - Store all data on both disks
- Robustness: very high, can withstand crashing
- Expensive solution
- Profit on random reads (load balancing)
- Write is slower than with one disk



Redundant Arrays of Independent Disks

RAID 3 & 4

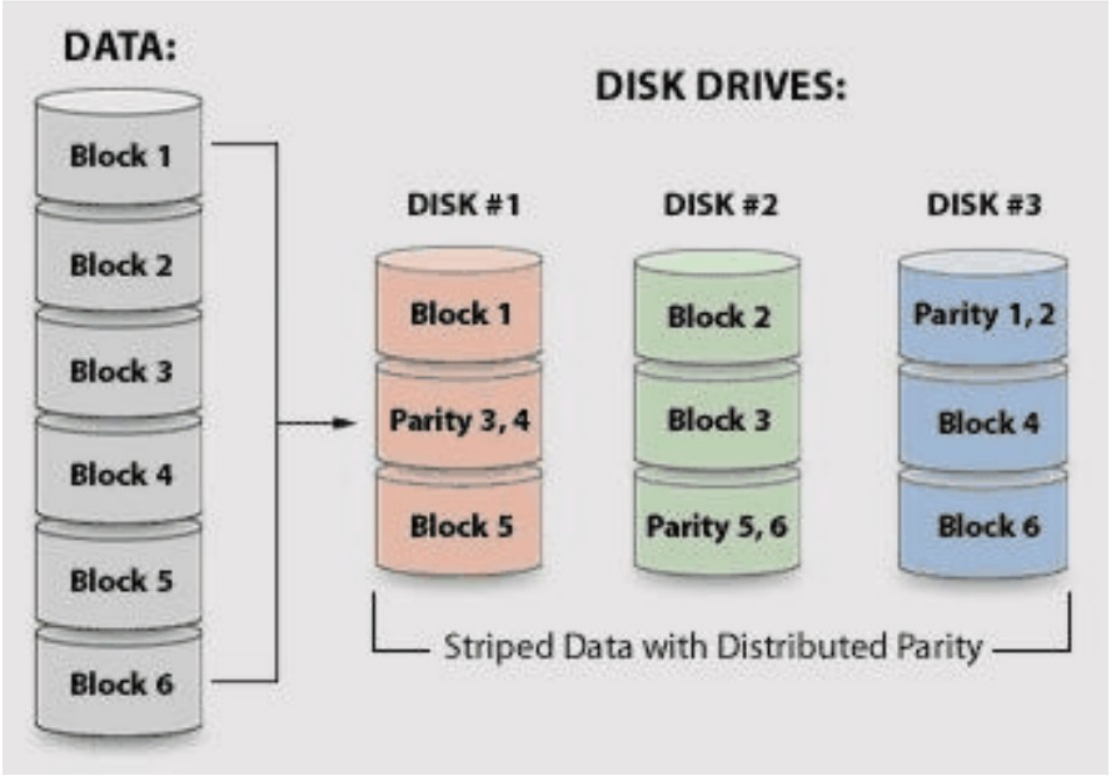
- RAID 0 (striping) over N disks + 1 extra disk for parity bits (so the sum is even)
- RAID 3: Synchronized disk heads
- RAID 3: short stripe
  - Fast sequential reads,
  - Parity on write mostly no read required
- RAID 4: long stripe
  - Random read fast
  - 1 write = 2 writes + 2 reads



Redundant Arrays of Independent Disks

RAID 5 & 6

- RAID 4 with parity information across all spread across all disks
- Better random read performance
- random write remains expensive
  (2 writes + 2 reads)
- Most popular RAID solution
- RAID 6: one extra parity disk



Redundant Arrays of Independent Disks

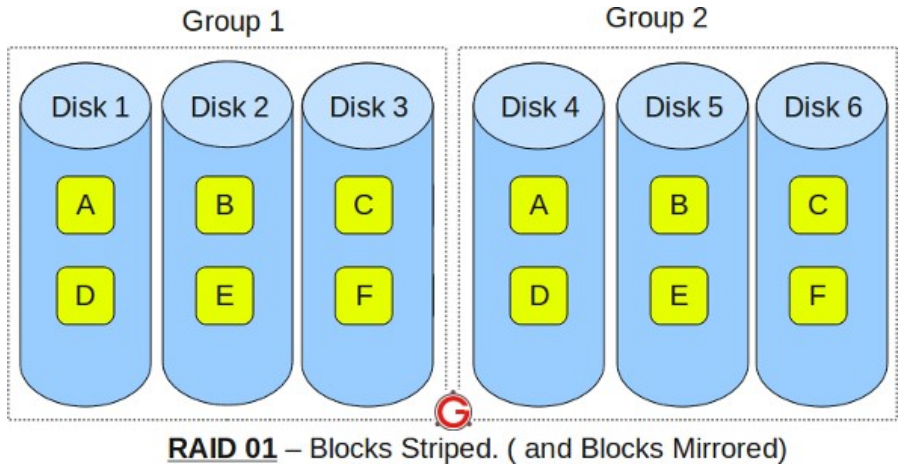
Combinations → RAID X+Y

- First organize data according to RAID Y
- Then we subdivide each of the obtained disks according to X
- Controller also applies both schemes in sequence (first Y then X)
- Notation: sometimes also X/Y, XY or even Y/X Y+X !

Redundant Arrays of Independent Disks

RAID 0+1 & 1+0

- RAID 0+1
  - Mirroring and then stripping
  - In read operation: select mirror first, then use striping

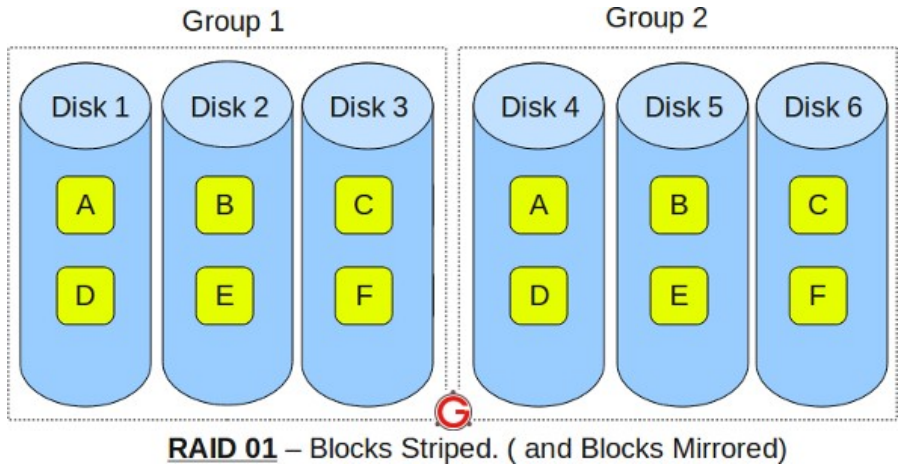


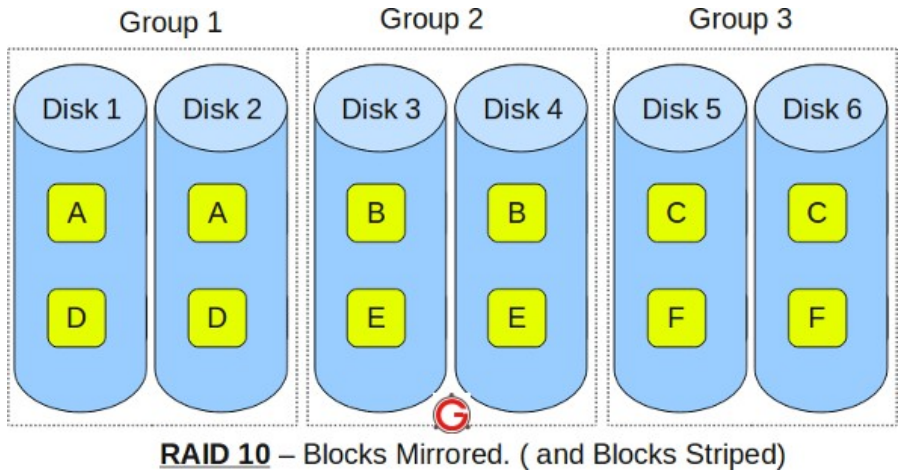
src: https://www.thegeekstuff.com/2011/10/raid10-vs-raid01/

Redundant Arrays of Independent Disks

RAID 0+1 & 1+0

- RAID 0+1
  - Mirroring and then stripping
  - In read operation: select mirror first, then use striping
- RAID 1+0
  - Striping and then duplicate
  - On read operation: striping and for each stripe we choose a mirror





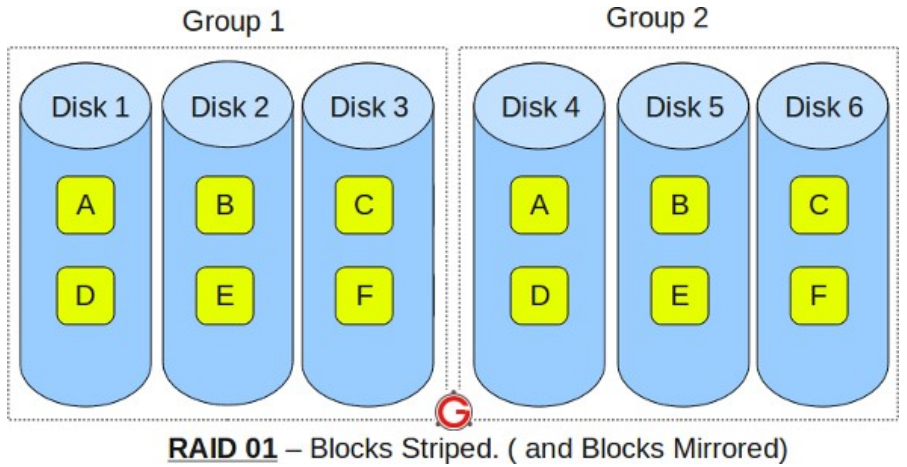
src: https://www.thegeekstuff.com/2011/10/raid10-vs-raid01/

Redundant Arrays of Independent Disks

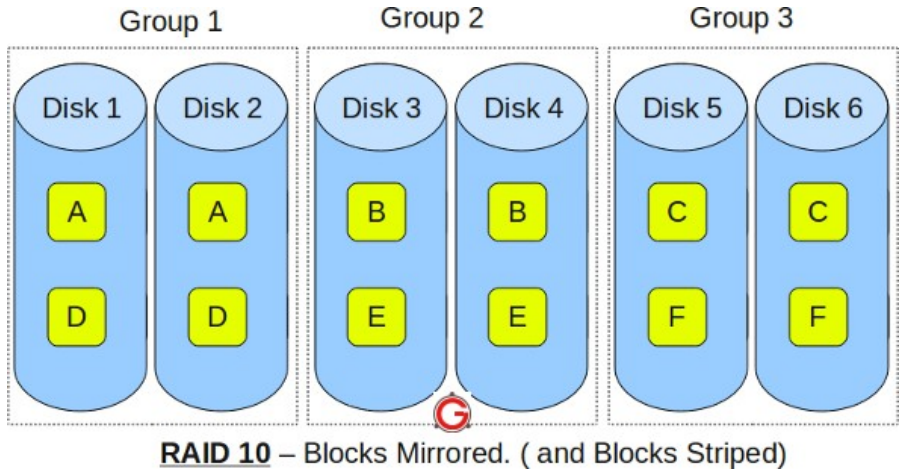
RAID 0+1 & 1+0

- RAID 0+1
  - Mirroring and then stripping
  - In read operation: select mirror first, then use striping
- RAID 1+0
  - Striping and then duplicate
  - On read operation: striping and for each stripe we choose a mirror

Comparison: RAID 1+0 is more robust, can handle many double disk failures many duplicate disk failures



RAID 01 – Blocks Striped. (and Blocks Mirrored)



RAID 10 – Blocks Mirrored. (and Blocks Striped)

src: https://www.thegeekstuff.com/2011/10/raid10-vs-raid01/

Key points

Disk Management

Pay attention to…

- Main concepts behind Disk Management
- Disk scheduling algorithms and their motivation
- RAID levels 0-6
- How to combine RAID levels, advantages and disadvantages of such a combination? Which RAID is best for certain applications (sequencial vs random reads)?

Further Reading

Essential: Disk Management

- OS Course Notes - chapter 6.
- Blackboard – Recorded lectures from Prof. Van Houdt.
  - Disk Management, deel I
  - Disk Management, RAID systemen

Questions?

Operating Systems

[1500WETOPS]

José Oramas